

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering



LAB MANUAL

DATA ENGINEERING LABORATORY

BTech CSE (III Year - 2nd Semester)

Prepared By:

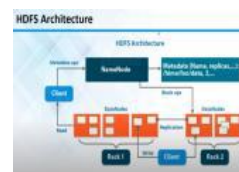
SOMANADHA SATYADEV BULUSU

Assistant Professor

Department of Computer Science and Engineering



$$P(A_i|B) = \frac{P(B|A_i) \cdot P(A_i)}{\sum_{j=1}^n P(B|A_j) \cdot P(A_j)}$$



Department of Computer Science and Engineering

VISION STATEMENT

To achieve academic and research excellence in essential technologies of Computer Science and Engineering by promoting a creative environment for learning and innovation

MISSION STATEMENT

1. To provide dynamic, innovative and flexible curriculum which equip the students with the necessary problem driven skills to strengthen their career prospects and potential to pursue higher studies.
2. To foster inquisitive-driven research among students and staff so as to reinforce the domain knowledge and address contemporary societal issues.
3. To inculcate ethical values, leadership qualities and professional behavior skills for improving the living standards of people

Table of Contents

<i>Laboratory Instructions</i>	4
<i>Course Objectives & Course Outcomes</i>	5
<i>Laboratory Syllabus</i>	6
<i>Bibliography:</i>	8
<i>Recommended Hardware & Software:</i>	8
<i>Laboratory Assessment</i>	9
<i>Course Based Projects (CBP)</i>	10
<i>Week 1: Basic Data Handling Commands (Use MPG Dataset)</i>	13
<i>Week 2: Standard Statistical Measures of Data</i>	14
<i>Week 3: Basic Plots for Data Exploration</i>	15
<i>Week 4: Data Pre-Processing Methods</i>	16
<i>Week 5: Feature Construction</i>	19
<i>Week 6: Feature Extraction</i>	22
<i>Week 7: HDFS(Storage)</i>	28
<i>Week 8 & 9: Map Reduce Programming</i>	30
<i>Week 10 & 11: Data Processing Tool – Hive</i>	35
<i>Week 12 & 13: MongoDB</i>	40

Laboratory Instructions



Do's

1. Read and understand how to carry out an experiment / activity thoroughly before coming to the laboratory.
2. Report any broken plugs or exposed electrical wires and any rectification needed, to your faculty/staff immediately.
3. Know the location of the fire extinguisher and the first aid box and have an understanding of how to use them in case of an emergency.
4. Report fires or accidents to your Faculty / Staff immediately.
5. Students have to come in formal wear and wear ID cards.
6. Should bring observation books and lab records for each lab session.



Don'ts

1. Do not open the system unit casing or monitor casing particularly unless you are agreed to do so and under supervision of Faculty / Staff. Some internal components hold electrical voltages of up to 30000 volts, which can be fatal.
2. Avoid stepping on electrical wires or any other computer cables.
3. Do not insert any material and particularly metal objects such as clips, pins and needles into computer casings. They may cause fire / electric shock.
- 5
4. Do not touch, connect or disconnect any plug or cable without your Faculty / Staff permission /assistance.
5. Do not remove anything from the computer laboratory without permission.
6. Do not disobey the instructions given and cause problems.
7. Do not eat or drink in the laboratory.
8. Any external storage device for loading / copying can be used only with prior permission from the lab-incharge.
9. Browsing internet for personal need during the Academic Schedules is prohibited

Course Objectives & Course Outcomes

COURSE OBJECTIVES:

- To explore and understand various data management and handling methods
- To understand the concept of data engineering
- To explore Hadoop framework and its components
- To use Big Data tools and techniques for data processing

COURSE OUTCOMES:

After completion of the course, students should be able to:

- CO-1: Implement data management and handling methods
- CO-2: Implement data engineering methods
- CO-3: Acquire the skills to work with Hadoop framework activities
- CO-4: Acquire the skills to work with NOSQL
- CO-5: Use various Big Data tools and techniques for data management

WEEK-1: Basic Data Handling Commands:

1. Read data from the csv file
2. Dimension of the data
3. Display data (top 5 rows and total data)
4. List the column names of a data frame
5. Change columns of a data frame
6. Display specific single column or multiple columns of a data frame
7. Bind sets of rows & Columns of dataframes
8. Find missing values in the dataset

Lab Instructions:

- Write python code & AutoMPG dataset to achieve above tasks using appropriate libraries.

WEEK-2: Standard Statistical Measures Of Data

1. Measures of central tendency – mean, median, mode
2. Measures of data spread
3. Dispersion of data – variance, standard deviation
4. Position of the different data values – quartiles, inter-quartile range (IQR).

Lab Instructions:

- Write python code & AutoMPG dataset to achieve above tasks using appropriate libraries.

WEEK-3: Basic Plots for Data Exploration (Use Iris dataset):

1. Generate box plot for each of the four predictors.
2. Generate box plot for a specific feature
3. Generate histogram for a specific feature
4. Generate Scatter plot of petal length vs. sepal length

Lab Instructions:

Write python code & Iris dataset to achieve above tasks using appropriate libraries.

WEEK-4: Data Pre-Processing Methods

1. Removing outliers / missing values.
2. Imputing standard values
3. Capping of values

Lab Instructions:

- Write python code & AutoMPG dataset to achieve above tasks using appropriate libraries.

WEEK-5: Feature Construction: (Use packages that are applicable)

1. Dummy coding categorical(nominal) variables.
2. Encoding categorical(ordinal) variables.
3. Transforming numeric(continuous) features to categorical features

Lab Instructions:

- Write python code & HomePrices dataset to achieve above tasks using appropriate libraries.

WEEK-6: Feature Extraction: (Use packages that are applicable)

1. Principal Component Analysis (PCA)
2. Singular Value Decomposition (SVD)
3. Linear Discriminant Analysis (LDA)
4. Feature Subset Selection

Lab Instructions:

- Write python code & Iris dataset to achieve above tasks using appropriate libraries.

WEEK-7: HDFS(Storage)

A. Hadoop Storage File System

Your first objective is to create a directory structure in HDFS using HDFS commands. Create the local files using Linux commands and move the files to HDFS directory and vice versa.

- i. Write a command to create the directory structure in HDFS.
- ii. Write a Command to move file from local unix/linux machine to HDFS.

B. Viewing Data Contents, Files and Directory

Try to perform these simple steps:

- i. Write HDFS command to see the contents of files in HDFS.
- ii. Write HDFS command to see contents of files which are present in HDFS.

Getting Files data from the HDFS to Local Disk

Your objective is to use HDFS commands to move data to HDFS for processing data. To process any data first move the data in HDFS. All files stored in HDFS can be accessed using HDFS commands.

- i. Write a HDFS command to copy the file from HDFS to local file system.

Lab Instructions:

- Install Hadoop and perform file management operations in Hadoop.

WEEK-8 & 9: Map Reduce Programming (Processing data).

Hadoop Map-Reduce framework is developed using Java, but the framework allows you to write programs in other languages as well.

Word Count:

The word count problem is the most famous using map reduce program. Same thing we can do with java but takes lot of time with huge file, in MR it will process less time even with huge and distributed files. The objective is to count the frequency of words of a large text.

Lab Instructions:

- Develop MapReduce example program in a MapReduce environment to find out the number of occurrences of each word in a text file

WEEK-10 & 11: Data Processing Tool – Hive (NoSQL query based language)

Hive command line tool allows you to submit jobs via bash scripts.

Identifying properties of a data set:

We have a table 'user data' that contains the following fields:

- *data_date: string*
- *user_id: string*
- *properties: string*

Lab Instructions:

- The properties field is formatted as a series of attribute=value pairs.Ex: Age=21; state=CA; gender=M;

WEEK-12 & 13: MongoDB

Create a database and explore basic database commands. Explore data types in MongoDB

Create a database with sample dataset and work with MongoDB Query Language

Lab Instructions:

- Install in a desktop machine and perform

WEEK-14: Internal Lab Examination

Bibliography:

TEXT BOOKS:

1. Machine Learning, Saikar Dutt, Subramanian Chandramouli, Amit Kumar Das, Pearson
2. Big Data and Analytics, Seema Acharya, Subhasini Chellappan, Wiley

REFERENCES:

1. Machine Learning, Tom M. Mitchell, McGraw-Hill Education

Recommended Hardware & Software:

Hardware:

- Intel based desktop PC with i3 or above CPU with at least 4GB RAM and 40GB free hard disk

Operating System:

- Any Flavor of WINDOWS currently supported by Micro Soft

Software

- Any Python editor such as VS Code, PyCharm, Jupyter Notebook, Google Colab e.t.c.
- Latest version of python with Machine Learning libraries
- JDK 1.8.0_202
- Hadoop 3.4.3
- Apache Hive 3.1.3
- MongoDB 4.4 or above

Laboratory Assessment

Internal Assessment (40 M):

- Staff should maintain the assessment register and Head of the department should monitor it

Day to day evaluation	10
Laboratory Record	10
Internal practical examination	10
Course Based Project (CBP)	10
Total	40

External Assessment (60 M):

- Exam conducted at the end of Lab
- One of the examiners will be appointed by controller of examinations in consultation with HOD

Initial procedures & diagrams	10
Conduct of experiment & collection of data	15
Analysis of data / results / graphs etc.	15
Course Based Project (CBP) Report	10
Viva-voce	10
Total	60

Course Based Projects (CBP)

1. Student Performance Prediction System

Problem Statement

- Build a model to predict student grades using demographic + academic data

Tasks:

- Handle missing/inconsistent data
- Feature construction (e.g., study hours × attendance)
- Feature selection (remove redundant variables)

Output:

- Clean dataset + feature importance report

2. Customer Segmentation using Behavioral Data

Problem Statement

- Cluster customers based on purchase patterns

Tasks:

- Data cleaning and transformation
- Dimensionality reduction (PCA)
- Identify key features driving clusters

Output:

- Segments + visualization dashboard

3. Fake Job Posting Detection

Problem Statement

- Detect fraudulent job listings

Tasks:

- Handle text + categorical data
- Feature extraction from text (TF-IDF or embeddings)
- Remove redundant features

Output:

- Binary classification + feature insights

4. Used Car Price Prediction

Problem Statement

- Predict resale value of cars

Tasks:

- Handle missing values, outliers
- Feature engineering (age of car, depreciation)
- Feature selection based on correlation

Output:

- Regression model + error analysis

5. Medical Data Preprocessing Pipeline

Problem Statement

- Clean and prepare patient data for downstream ML

Tasks:

- Handle noisy, missing, inconsistent data
- Normalize numerical features
- Encode categorical features

Output:

- Reusable preprocessing pipeline

6. A/B Testing Framework for Website Optimization

Problem Statement

- Compare two versions of a web page

Tasks:

- Hypothesis testing (t-test / z-test)
- Confidence intervals
- Sampling techniques

Output:

- Statistical decision + visualization

7. Monte Carlo Simulation for Stock Price Prediction

Problem Statement

- Simulate future stock prices using probabilistic models

Tasks:

- Use Gaussian distribution
- Generate multiple simulations
- Analyze variance and expected returns

Output:

- Simulation plots + risk analysis

8. Disease Spread Simulation using Probability Models

Problem Statement

- Model spread using Poisson/Binomial distributions

Tasks:

- Define random variables
- Simulate infection spread
- Analyze correlation between variables

Output:

- Graphs showing spread scenarios

9. Central Limit Theorem Demonstration Tool

Problem Statement

- Build a tool to demonstrate CLT

Tasks:

- Sample from different distributions
- Show convergence to normal distribution

Output:

- Interactive visualization

10. Feature Correlation Analyzer

Problem Statement

- Analyze relationships between variables in datasets

Tasks:

- Compute covariance, correlation
- Detect multi collinearity
- Suggest feature removal

Output:

- Correlation heatmap + recommendations

11. Log Analysis using Hadoop

Problem Statement

- Analyze server logs for usage patterns

Tasks:

- Store logs in HDFS
- Process using MapReduce
- Extract insights (peak usage, errors)

Output:

- Aggregated metrics

12. Movie Recommendation using Hadoop Ecosystem

Problem Statement

- Build a recommendation system

Tasks:

- Use large dataset
- Process with Hadoop/Spark
- Compute similarity between users/items

Output:

- Top-N recommendations

13. Data Lake vs Data Warehouse Comparative Study

Problem Statement

- Implement both approaches for same dataset

Tasks:

- Store structured vs unstructured data
- Query performance comparison
- Storage efficiency analysis

Output:

- Comparative report

14. E-commerce Backend using MongoDB

Problem Statement

- Design backend for product catalog

Tasks:

- Schema design (flexible documents)
- CRUD operations
- Aggregation queries (sales, trends)

Output:

- Working backend + API

15. Social Media Analytics Dashboard

Problem Statement

- Store and analyze social media posts

Tasks:

- Use MongoDB for storing posts
- Perform aggregation (likes, shares)
- Handle arrays and nested data

Output:

- Analytics dashboard

Week 1: Basic Data Handling Commands (Use MPG Dataset)

1. Reading and Displaying Data

Importing pandas and reading the dataset:

```
import pandas as pd
df = pd.read_csv("mpg_data.csv")
df
```

2. Display Top 5 Rows

```
df.head(5)
```

3. Dimension of Data

```
df.shape
```

4. List Column Names

```
df.columns
```

5. Rename Columns

```
df.rename(columns = {'car name': 'Cars Name'}, inplace=True)
df.head()
```

6. Specific Column Selection

#Single Column:

```
df.mpg.head()
```

#Multiple Columns:

```
df[["mpg", "origin"]].head()
```

7. Bind sets of rows of dataframes

```
df1 = pd.DataFrame({'a': [1, 2], 'b': [3, 4]})
df2 = pd.DataFrame({'a': [5, 6], 'b': [7, 8]})
```

```
result = pd.concat([df1, df2])
print(result)
```

8. Bind sets of columns of data frames. (Concat)

```
df1 = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
df2 = pd.DataFrame({'c': [7, 8, 9], 'd': [10, 11, 12]})
df3 = pd.concat([df1, df2], axis=0, sort=False)
print(df3)
```

9. Missing Values

```
df.isnull().count()
```

Week 2: Standard Statistical Measures of Data

1. Measures of central tendency – mean, median, mode

```
import pandas as pd
import numpy as np

#reading the data
df = pd.read_csv("mpg_data.csv")
df

#Mean
df['displacement'].mean()

#Median
df['displacement'].median()

#Mode
df['displacement'].mode()

#count of each value in the col
df['displacement'].value_counts()
```

2. Measures of data spread

```
df.describe()
```

3. Dispersion of data – variance, standard deviation

```
# standard deviation for each column
df.std()

#Varaince of all columns
columns = ['mpg', 'cylinders', 'displacement']
df.var()
```

4. Position of the different data values – quartiles, inter-quartile range (IQR).

```
q1 = np.percentile(df['displacement'], 25, interpolation='midpoint')
q1

q2 = np.percentile(df['displacement'], 50, interpolation='midpoint')
q2

q3 = np.percentile(df['displacement'], 75, interpolation='midpoint')
q3

# Inter quartile Range
IQR = q3-q1
IQR

df['displacement'].quantile(0.25)
```

Week 3: Basic Plots for Data Exploration

1. Generate box plot for each of the four predictors.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#from statistics import median
#pip install matplotlib==3.2.2

df=pd.read_csv('iris.csv')
df.head()

#Box plot for PetalLength
sns.boxplot(x='Species',y='PetalLengthCm',data=df)
plt.show()

#Box plot for SepalLengthCm
sns.boxplot(x='Species',y='SepalLengthCm',data=df)
plt.show()

#Box plot for SepalWidthCm
sns.boxplot(x='Species',y='SepalWidthCm',data=df)
plt.show()

#Box plot for PetalWidthCm
sns.boxplot(x='Species',y='PetalWidthCm',data=df)
plt.show()
```

Observation:

- Outliers are observed in petal Length for Setosa and Versicolor feature
- Outliers are observed in Sepal Length and Sepal Width feature for Virginica
- Outliers are observed in Petal Width for Setosa

2. Generate box plot for a specific feature

```
#Box plot for SepalLengthCm
sns.boxplot(x='Species',y='SepalLengthCm',data=df)
plt.show()
```

3. Generate histogram for a specific feature

```
sns.histplot(x='SepalWidthCm',data=df)
plt.show()

sns.hist(x='SepalWidthCm',data=df)
plt.show()
```

4. Generate Scatter plot of petal length vs. sepal length

```
df.plot(kind="scatter",x='PetalLengthCm',y='SepalLengthCm')
#plt.show()
plt.grid()
sns.scatterplot(x='PetalLengthCm',y='SepalLengthCm',data=df)
```

Week 4: Data Pre-Processing Methods

1. Removing outliers / missing values.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statistics import median
import numpy as np

df = pd.read_csv('mpg_data.csv')
df.head(5)

sns.boxplot(x=df['mpg'], data=df)

df.loc[df['horsepower'] == '?']
df.isnull().sum()

# replace '?' with NaN
df.replace('?', np.nan, inplace=True)
# df.loc[df['horsepower'] == '?'] , shows that '?' is replaced
df.isnull().sum()
```

The IQR method is a simple and effective way to remove outliers from a dataset. It works by identifying data points that fall outside of the range of $Q1 - 1.5 * IQR$ and $Q3 + 1.5 * IQR$, where $Q1$ and $Q3$ are the first and third quartiles of the data.

```
# First and Third Quartile
q1 = df['mpg'].quantile(0.25)
q3 = df['mpg'].quantile(0.75)

# IQR
IQR = q3 - q1

# calculate lower limit and upper limit
LL = q1 - 1.5*IQR
UL = q3 + 1.5*IQR

print(q1, q3, IQR, LL, UL)

# Identifying outliers
outliers = df[(df['mpg'] <= LL) | (df['mpg'] >= UL)]

# Remove Outliers
df_outliers_removed = df.drop(outliers.index)
```

The Z-score method is another common method for removing outliers. It works by calculating the number of standard deviations each data point is away from the mean. Data points with Z-scores greater than a certain threshold (typically 3 or 4) are considered outliers.

```
# Calculate the Z-score
z_scores = (df['mpg'] - df['mpg'].mean()) / df['mpg'].std()
```



```

# Identify outliers
outliers = df[np.abs(z_scores) > 3]

# Remove outliers
df_outliers_removed = df.drop(outliers.index)

# Drop rows with missing values
df_na_rows_removed = df.dropna()

# Drop columns with missing values
df_na_columns_removed = df.dropna(axis=1)

# Impute missing values with the median value of the column
df['mpg'].fillna(df['mpg'].median(), inplace=True)

```

2. Inputing standard values

```

# Box plot before replacing outliers with median
sns.boxplot(x='acceleration', data=df)

# quartiles for acceleration
q1 = df['acceleration'].quantile(0.25)
q3 = df['acceleration'].quantile(0.75)
median = df['acceleration'].quantile(0.50)
IQR = q3 - q1

# calculating Lower limit and upper limit
LL = q1 - 1.5*IQR
UL = q3 + 1.5*IQR

# Outliers
outliers_acc = df[(df['acceleration'] <= LL) | (df['acceleration'] >= UL)]
outliers_acc

#replacing outliers with median values using indexing method.
# Convert target values into array
arr = df['acceleration'].values

# if arr values are in between LL and UL, represent that
# value as True else as False
# False values are outliers
true_index = (arr > LL) & (arr < UL)
true_index

# Outliers are represented as False
# let us substitute mean value in the place of outliers
# so calculate mean of all values other than the outliers.
mid1 = np.mean(df['acceleration'][true_index])
mid1 15.501542416452427

#replace False index value by mean
false_index = ~true_index

# ~ is bool operator for inverse
df['acceleration'].values[false_index] = mid1
df

```

3. Capping of values

There are a number of different ways to cap values. A common approach is to set a maximum and minimum value for the column. Any values that fall outside of this range are replaced with the maximum or minimum value, respectively.

Another approach is to use a percentile-based capping method. This method sets the maximum and minimum values for the column to the percentiles of the data. For example, you could set the maximum value to the 95th percentile, and the minimum value to the 5th percentile. This would remove any values that fall outside of the normal range of the data.

```
df_cap = pd.read_csv('mpg_data.csv')
df_cap.shape

# Identifying Outliers in cylinders
max_threshold = df_cap['acceleration'].quantile(0.95)
min_threshold = df_cap['acceleration'].quantile(0.05)
print(max_threshold, min_threshold)

# Before capping, box plot
sns.boxplot(x='acceleration', data=df_cap)

# Or # Cap the 'price' column at min - max threshold
df_cap['acceleration'] = df_cap['acceleration'].apply(
    lambda x: np.clip(
        x, df_cap['acceleration'].quantile(0.05),
        df_cap['acceleration'].quantile(0.95)))

# After capping, box plot
sns.boxplot(x='acceleration', data=df_cap)
```

Week 5: Feature Construction

1. Dummy coding categorical(nominal) variables.

Nominal data is categorical data that is qualitative, meaning that it does not have any inherent order. E.g Customer satisfaction ratings (e.g., 1-5 stars) ,Educational level (e.g., high school diploma, bachelor's degree, master's degree) ,Military rank (e.g., private, corporal, sergeant)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, OrdinalEncoder

df = pd.read_csv('homeprices.csv')
df

# As town is categorical
dummies = pd.get_dummies(df.town)
dummies

# Avoid dummy variable trap, by dropping one column
dummies = dummies.drop(['robinsville'],axis=1)
dummies

# Add dummies to the original df
merged= pd.concat([df, dummies], axis=1)
merged

# drop town
final = merged.drop(['town'], axis=1)
final

# Create a OneHotEncoder object
# the encoder will simply ignore the unknown categories and will not create any new features for them.
x.toarray() = OneHotEncoder(handle_unknown='ignore')

# Fit the encoder to the data, Transform the data
df_encoded = encoder.fit_transform(df[['town']])

# Print the one-hot encoded data
pd.DataFrame(df_encoded.toarray())
```

Here are some of the benefits of using one-hot encoding:

- It makes it easy to use categorical variables in machine learning models.
- It preserves the information in the categorical variables.
- It is easy to understand and implement.

However, there are also some limitations to using one-hot encoding:

- It can increase the dimensionality of the data.
- It can be sensitive to outliers.
- It can be difficult to interpret the results of machine learning models that use one-hot encoded variables.

When to use one-hot encoding vs dummy encoding:

- One-hot encoding is generally preferred over dummy encoding, as it is more efficient and less prone to overfitting.
- Dummy encoding should be used when the number of categories in the categorical variable is small and the categories have a natural order

2. Encoding categorical(ordinal) variables

Ordinal data is categorical data that is quantitative, meaning that it has a natural order. E.g. Eye color, Hair color, Nationality, Religion, Political affiliation

Label Encoder

A label encoder is a machine learning technique used to convert categorical variables into numerical variables. It is a simple and effective way to prepare data for machine learning models, as most machine learning models only accept numerical data.

```
iris = pd.read_csv('iris.csv')
iris
```

```
encoder = LabelEncoder()
iris_encoded = encoder.fit_transform(iris['Species'])
print(iris_encoded)
iris.Species = iris_encoded
iris
```

Observation:

- Species attribute is the ordinal attribute, so it needs to be converted into numeric
- 0 -> Iris-setosa, 1 -> Iris-versicolor, 2 -> Iris-virginica

Benefits of using label encoders:

- They are simple to understand and implement.
- They are efficient and scalable.
- They preserve the information in the categorical variable.

Limitations of using label encoders:

- They can introduce an artificial order to the categorical variables.
- They can be sensitive to outliers.
- They can be difficult to interpret the results of machine learning models that use label encoded variables.

3. Transforming numeric(continuous)features to categorical features

Binning: This involves dividing the numeric feature into a number of bins, and then assigning each value in the feature to a bin. The bins can be created using equal intervals, or by using a more sophisticated method such as quantile binning

```
plt.hist(iris.PetalWidthCm, edgecolor='w')
plt.show()
```

```
iris['petal_category'] = pd.cut(
    iris.PetalWidthCm,
    bins=[0, 1, 2, 2.5],
    labels=['small', 'medium', 'large'])
```

```
print(iris['petal_category'])
iris['petal_category'].value_counts()
```

Ordinal Encoder : An ordinal encoder is a machine learning technique used to convert ordinal categorical variables into numerical variables. Ordinal categorical variables are categorical variables that have a natural order, such as customer satisfaction ratings (e.g., 1-5 stars) or educational level (e.g., high school diploma, bachelor's degree, master's degree).

```
encoder = OrdinalEncoder()  
iris['petal_category'] = encoder.fit_transform(iris['petal_category'])  
iris
```

Observation:

- The attribute PetalLengthCm is converted into Ordinal type and then it is encoded into continuous data
- Scikitlearn Ordinal Encoder is used to transform categorical data into continuous data

Benefits of using ordinal encoders:

- They are simple to understand and implement.
- They are efficient and scalable.
- They preserve the information in the ordinal categorical variable.
- They can be used with machine learning models that expect numerical features.

Limitations of using ordinal encoders:

- They can be sensitive to outliers.
- It can be difficult to interpret the results of machine learning models that use ordinal encoded variables.

Week 6: Feature Extraction

1. Principal Component Analysis (PCA)

```
# importing all the packages
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import sklearn.preprocessing
import numpy as np
df = pd.read_csv('iris.csv')
iris = df
df

# Determining initial dimensions of the data
x = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = df[['Species']]
print(x.head())
print(y.head())
print(x.shape, y.shape)

# Define a color mapping for the classes
color_mapping = { 'Iris-setosa': 'red', 'Iris-versicolor': 'green', 'Iris-virginica': 'blue' }

# Map the 'Species' column to colors
colors = y['Species'].map(color_mapping)

# Scatter plot with colors plt.scatter(x.iloc[:, 0], x.iloc[:, 1], c=colors)
plt.show()

pca = PCA()
X_new = pca.fit_transform(x)
cov_mat = pca.get_covariance()
cov_mat

eig_vals, eig_vec = np.linalg.eig(cov_mat)
print("Eigen Values :\n", eig_vals)
print("Eigen Vectors: \n", eig_vec)

# PCA for target dimension on the dataset
pca = PCA(n_components=2)
pca.fit(x)

PCA(n_components=2)
pca.components_

# Transforming the dataset from 4 dimensions to 2 dimensions using PCA
z = pca.transform(x)
z.shape
plt.scatter(z[:, 0], z[:, 1], c=colors)
plt.show()
```

```
# Variance ratio of the target dimensions pca.explained_variance_ratio_
```

Observation:

- The three classes are well separated.
- The first principal component contains 92.46% of the variance and
- the second principal component contains 5.3% of the variance.
- Together, the first two principal components contain 97.76% of the information.
- The third and fourth principal component contained the rest of the variance of the dataset.

2. Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a fundamental matrix factorization technique used in linear algebra, data analysis, and machine learning. It decomposes a given matrix into three constituent matrices, which can be thought of as capturing essential information about the original matrix. In SVD, **any $m \times n$** matrix can be expressed as the product of three matrices:

U ($m \times m$), Σ ($m \times n$), and V^T ($n \times n$)

where

- U and V^T are orthogonal matrices
- Σ is a diagonal matrix containing singular values.

These **singular values in Σ** represent the **magnitude of information** in the data along each dimension and can be used for dimensionality reduction, noise reduction, and data compression.

SVD finds applications in a wide range of fields, including data compression, image processing, natural language processing, and recommendation systems. It is particularly useful in reducing the dimensionality of data, which is beneficial for feature selection, removing noise, and uncovering latent structures in large datasets. Additionally, SVD plays a crucial role in other matrix factorization techniques, such as Principal Component Analysis (PCA) and collaborative filtering in recommendation systems.

```
iris = x.astype(float)
U, S, Vt = np.linalg.svd(iris, full_matrices=False)
print(f"{U.shape}\n{S}\n{Vt}")
```

NOTE: **`numpy.linalg.svd`** actually returns a Σ that is not a diagonal matrix, but a list of the entries on the diagonal.

```
# After SVD
```

```
import matplotlib.pyplot as plt
```

```
# Plot the first two columns of U
plt.scatter(U[:, 0], U[:, 1], c=colors)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

3. Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) is a dimensionality reduction and classification technique commonly used in machine learning and pattern recognition. It aims to find a linear combination of features that maximizes the separation between multiple classes while minimizing the variance within each class. LDA is particularly useful for supervised learning tasks where the goal is to classify data into predefined categories. By transforming the data into a lower-dimensional space, LDA not only reduces dimensionality but also enhances the discriminatory power of the features, making it easier for classifiers to distinguish between different classes. LDA is widely used in applications like face recognition, document classification, and bio informatics.

Unlike Principal Component Analysis (PCA), which focuses on capturing overall variance in the data, LDA focuses on maximizing class separability. It achieves this by finding the linear projections of the data that best separate the classes, making it a valuable tool for feature selection and classification tasks where class discrimination is crucial.

```
lda = LinearDiscriminantAnalysis(n_components=2)
X_r2 = lda.fit(x, y.values.ravel()).transform(x)

# 'y' variable should be reshaped into a 1D array instead of a
# column-vector. ravel() function is used to flatten 'y' into a 1D array

#Getting the variance ratio
lda.explained_variance_ratio_

# visualizing 2D datab in the form of scatter plot
plt.scatter(X_r2[:,0],X_r2[:,1],c=colors)
```

Observation:

- LDA is able to separate the classes very well after dimensionality reduction

4. Feature Subset Selection

Feature subset selection is a process of choosing a subset of the most relevant features (variables) from your dataset while eliminating less important or redundant ones. This is done to improve model performance, reduce overfitting, and enhance interpretability. There are various techniques for feature subset selection

4.1 Correlation Matrix with Heatmap:

- Correlation states how the features are related to each other or the target variable.
- Correlation can be positive (increase in one value of feature increases the value of the target variable) or negative (increase in one value of feature decreases the value of the target variable)
- The logic behind using correlation for feature selection is that good variables correlate highly with the target. Furthermore, variables should be correlated with the target but uncorrelated among themselves.
- If two variables are correlated, we can predict one from the other. Therefore, if two features are correlated, the model only needs one, as the second does not add additional information.
- Heatmap makes it easy to identify which features are most related to the target variable, we will plot heatmap of correlated features using the seaborn library.

```
# Create a correlation matrix
correlation_matrix = df.corr()

# Create a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap of Iris Dataset Features")
plt.show()
```



```

# Load your dataset, assuming it's in a DataFrame named 'df'
# Replace 'your_target_column'('species') with the name of your target variable column
# Remove the target column from the DataFrame to focus on feature-feature correlations
X = df.drop('Species', axis=1)

# Calculate the correlation matrix
correlation_matrix = X.corr()

# Create a heatmap to visualize the correlations
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Feature Correlation Heatmap")

# Define a correlation threshold (e.g., 0.7) above which features are considered highly correlated
correlation_threshold = 0.7

```

#Find highly correlated feature pairs

```

highly_correlated_features = set()
for i in range(len(correlation_matrix.columns)):
    for j in range(i):
        if abs(correlation_matrix.iloc[i, j]) > correlation_threshold:
            colname = correlation_matrix.columns[i]
            highly_correlated_features.add(colname)

```

```

# Print the highly correlated feature pairs
print("Highly Correlated Features:")
print(highly_correlated_features)

```

```

# Drop one feature from each correlated pair
X_filtered = X.drop(columns=highly_correlated_features)

```

```

# Now, X_filtered contains the dataset with highly correlated features removed

```

4.2: Feature Importance(wrapper Approach)

You can get the feature importance of each feature of your dataset by using the feature importance property of the model. Feature importance gives you a score for each feature of your data, the higher the score more important or relevant is the feature towards your output variable. Feature importance is an inbuilt class that comes with Tree Based Classifiers, we will be using Extra Tree Classifier for extracting the top 10 features for the dataset.

```

import numpy as np
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load the Iris dataset
data = load_iris()
X = data.data # Features
y = data.target # Target variable

```

```

# Split the data into a training and testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an ExtraTreesClassifier
clf = ExtraTreesClassifier(n_estimators=100, random_state=42)

# Fit the classifier to the training data
clf.fit(X_train, y_train)

# Calculate feature importance
feature_importances = clf.feature_importances_

# Sort features by importance in descending order
sorted_indices = np.argsort(feature_importances)[::-1]

# Select the top 'k' most important features (e.g., top 2)
k = 2
selected_feature_indices = sorted_indices[:k]

# Select the corresponding features from the original data
X_train_selected = X_train[:, selected_feature_indices]
X_test_selected = X_test[:, selected_feature_indices]

# Train and evaluate a classifier using the selected features
clf_selected = ExtraTreesClassifier(n_estimators=100, random_state=42)
clf_selected.fit(X_train_selected, y_train)
y_pred = clf_selected.predict(X_test_selected)

# Calculate accuracy with the selected features
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy with {k} selected features: {accuracy:.2f}")

# Calculate feature importances
feature_importances = clf.feature_importances_

# Get feature names
feature_names = data.feature_names

# Create a bar graph to visualize feature importances
plt.figure(figsize=(10, 6))
plt.bar(range(len(feature_importances)), feature_importances, tick_label=feature_names)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importances")
plt.xticks(rotation=45)
plt.show()

```

4.3: Univariate Selection(Filter approach)

Univariate feature selection is a filter-based approach for feature subset selection. It assesses the statistical relationship between each feature and the target variable independently, without considering interactions between features. One common technique for univariate feature selection is using the SelectKBest class from scikit-learn. It selects the top k features with the highest scores based on a specified statistical test.

```
# Import necessary libraries
import numpy as np
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

# Load a sample dataset (Iris dataset in this example)
data = load_iris()
X = data.data # Features
y = data.target # Target variable
feature_names = data.feature_names

# Create SelectKBest object and specify the number of features to select (e.g., 2)
k = 2
selector = SelectKBest(score_func=chi2, k=k)

# Fit the selector to the data and transform the features
X_new = selector.fit_transform(X, y)

# Get the indices of the selected features
selected_feature_indices = selector.get_support(indices=True)

# Get the column names of the selected features
selected_feature_names = [feature_names[i] for i in selected_feature_indices]

# Print the column names of the selected features
print("Column names of selected features:", selected_feature_names)
```

Week 7: HDFS(Storage)

Running HDFS commands on a Windows machine depends entirely on how **Hadoop** environment is set up. Since HDFS is natively built for Linux, Windows users typically interact with it in one of three ways: **WSL2 (Windows Subsystem for Linux)**, **PowerShell/CMD** (with WinUtils), or a **GUI**. Here is how to perform those same operations on Windows using **PowerShell/CMD (Standard Setup)**

1. Prerequisites

- Install Java (JDK 8 or 11) as Hadoop is Java-based: <https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>
- Recommended location to extract is: C:\java8
- Windows 10/11 (64-bit)

2. Download Hadoop

- Download latest stable version from the following link : <https://www.apache.org/dyn/closer.cgi/hadoop/common/hadoop-3.4.3/hadoop-3.4.3.tar.gz>
- Recommended location to extract is: "C:\hadoop"
- Use following command to extract the above file: `tar -xvzf hadoop-3.4.3.tar.gz`

3. Setup Environment Variables

- In the System Environment Variable, Set **JAVA_HOME** as "C:\java8" (or whatever it is)
- In the System Environment Variable, Set **HADOOP_HOME** as "C:\hadoop" (or whatever it is)
- Add following to "PATH" system variable %HADOOP_HOME%\bin, %HADOOP_HOME%\sbin

4. Add winutils.exe, hadoop.dll, hdfs.dll

- Download "winutils.exe", "hadoop.dll", "hdfs.dll" files from the following Link: <https://github.com/robquillarr/spark-winutils-3.3.1/tree/master/hadoop-3.4.0-win10-x64/bin>
- Copy those three files into the "C:\hadoop\bin" directory. i.e. %HADOOP_HOME%\bin

5. Configure Hadoop

- Go to C:\hadoop\etc\hadoop i.e. %HADOOP_HOME%\etc\hadoop
- Update following code into the **core-site.xml**:

```
<configuration>
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
</property>
</configuration>
```

- Update following code into the **hdfs-site.xml**:

```
<configuration>
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
</configuration>
```

- Copy "mapred-site.xml.template" to "mapred-site.xml"

- Update following code into the **mapred-site.xml**:

```
<configuration>
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
</configuration>
```

- Update following code into the **yarn-site.xml**:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

6. Format Namenode

- Format Namenode by running following command: **"hdfs namenode -format"**

7. Start Hadoop

- Run following commands at the command prompt
 - start-dfs.cmd
 - start-yarn.cmd

8. Verify Installation

Service	URL
HDFS	http://localhost:9870
YARN	http://localhost:8088

9. Experiments For Week 7

- **Hadoop Storage File System**
 - Write a command to create the directory structure in HDFS.
 - ◆ **hdfs dfs -mkdir -p /user/hadoop/data/input**
 - -mkdir → creates directory
 - -p → creates parent directories automatically (like Unix mkdir -p)
 - Example structure created:
 - /user/hadoop/data/input
 - Write a Command to move file from local unix/linux/windows machine to HDFS.
 - ◆ **hdfs dfs -put C:\data\sample.txt /user/hadoop/data/input/**
 - (or)
 - ◆ **hdfs dfs -copyFromLocal C:\data\sample.txt /user/hadoop/data/input/**
 - **Viewing Data Contents, Files and Directory**
 - Write HDFS command to see the contents of files in HDFS
 - ◆ **hdfs dfs -ls /user/hadoop/data/input**
 - Write HDFS command to see contents of files which are present in HDFS
 - ◆ For entire content: **hdfs dfs -cat /user/hadoop/data/input/sample.txt**
 - ◆ For the first few lines: **hdfs dfs -head /user/hadoop/data/input/sample.txt**
 - ◆ For the last few lines: **hdfs dfs -tail /user/hadoop/data/input/sample.txt**
 - **Getting Files data from the HDFS to Local Disk**
 - **hdfs dfs -get /user/hadoop/data/input/sample.txt C:/data/**
 - ◆ At the command prompt type following : **type C:\data\sample.txt**

Week 8 & 9: Map Reduce Programming

1. What is Map-Reduce?

- **MapReduce is a programming model for processing large datasets in parallel by splitting tasks into smaller units (Map) and then combining results (Reduce).** It is introduced by Google and implemented in Apache Hadoop using Java. In Hadoop, it works on top of Hadoop Distributed File System (HDFS), by distributing computations across cluster of nodes for scalability, fault tolerance, and efficiency.

2. When to use Map-Reduce

- Map-Reduce is best for problems that can be expressed as:
 - **Split Data → Process in parallel → Aggregate results**
- Map-Reduce can be used in problems areas like:
 - **Counting & Aggregation:** Word count, Number of users per country, Total sales per region
 - **Log Processing & Analytics:** Website traffic analysis, Error log counting, User activity tracking
 - **Search & Indexing:** Building inverted index, Keyword → document mapping
 - **Data Transformation (ETL):** Convert raw logs → structured format, Filtering unwanted data, Data cleaning
 - **Sorting & Filtering:** Sort large datasets, Remove duplicates, Filter records (e.g., only users > age 18)
 - **Join Operations:** Join user table + transaction table, Relational joins
 - **Machine Learning (Basic):** Naive Bayes, K-means (limited efficiency), Feature counting
 - **Graph Processing (Basic Level):** Page Rank (initial implementations), Connectivity analysis
 - And in many more areas ...

3. When NOT to use Map-Reduce:

- Following are the problem areas where usage of Map-Reduce is not recommended
 - Map-Reduce may not be used in **Real-time processing scenarios** due to its high latency
 - Due to disk I/O operations of Map-Reduce, it may not be used in **Iterative algorithms**
 - When there is only **small amount of data**, using Map-Reduce is a overkill
 - For **complex graph algorithms**, usage of Map-Reduce is inefficient

4. Key Features of Map-Reduce

- Following are the key features of Map-Reduce, benefit many data processing application
 - **Task Parallelism:** Tasks run simultaneously across nodes.
 - **Scalability:** Handles petabytes of data by adding more nodes.
 - **Data Locality:** Computation happens where data resides, reducing network overhead.
 - **Fault Tolerance:** Automatic recovery from node failures.

5. Key Components

- **ResourceManager (RM):** Global master for resource allocation. Decides how much CPU/memory each application gets. Does not run tasks itself.
- **NodeManager (NM):** Runs on each worker node. Reports resource availability to RM. Manages containers (isolated environments for tasks).
- **ApplicationMaster (AM):** Created per application (e.g., a MapReduce job). Handles job-specific scheduling and execution. Requests resources from RM and launches tasks in containers via NM.
- **Containers:** Lightweight execution environments (like mini virtual machines). Run tasks with allocated CPU/memory.

6. How MapReduce Works in Hadoop:

- In Apache Hadoop, MapReduce runs on top of:
 - **Hadoop Distributed File System (HDFS)** → Storage and Splitting files into data blocks
 - **Yet Another Resource Negotiator (YARN)** → Resource management
- **Execution Flow**
 - **Data Storage:** User stores input data as files in **HDFS** for the required map-reduce processing.

- **Job Submission:** A . The JobTracker (master) coordinates execution, while TaskTrackers (workers) run tasks on cluster nodes.
- **Data Split:** When a client submits a job (JAR file with Mapper & Reducer code) for map-reduce on the data, then the user data files get split into several data chunks (default size is 128 MB) called **InputSplits**.
- **Resources Allocation:** YARN assigns resources to different tasks that are distributed across different Data Nodes, where each task performs same mapping job with the data chunk given to it
- **Map Task(s) :** Each mapper task runs close to data i.e. in a data node. All these data nodes perform same mapping jobs in parallel on their respective data chunks produces outputs as **key-value** pairs
- **Shuffle & Sort:** Intermediate results are shuffled and sorted by the key automatically in each respective node locally
- **Reduce Task(s):** Reducer jobs (0,1 or more) aggregates all the sorted outputs from different nodes and grouped values for each key
- **Final Output:** Final output is stored back in HDFS. Failed tasks are automatically re-executed on other nodes

7. Word Count Example:

- **Input Data:** `hello world hello`
- **Map Output:** `(hello,1), (world,1), (hello,1)`
- **Shuffle Output:** `hello → [1,1], world → [1]`
- **Reduces Output:** `(hello,2), (world,1)`

Experiment 8 & 9:

1. Problem: Simple Word Count Program Using Map Reduce Java API:

- **Word Count:** The word count problem is the most famous using map reduce program. Same thing we can do with java but takes lot of time with huge file, in MR it will process less time even with huge and distributed files. The objective is to count the frequency of words of a large text.
- Lab Instructions: Develop MapReduce example program in a MapReduce environment to find out the number of occurrences of each word in a text file

2. Initial Setup:

- Open a command window in Administrator mode
- Check if JRE 8.0 or above is installed using command : `javac -version`
- Create a new directory called **wordcount**, where ever you want and move to that directory
- Create a new text file in and rename it as **WordCount.java**

3. Map-Reduce Java program:

- Open **WordCount.java** file and type following program. For the sake of easy understanding, entire program is divided into multiple blocks and each block is explained with a comment heading:
- Observe that there are packages with name containing “hadoop” or “mapreduce” in them

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

- Word Count class begins here. This class contains 2 inner classes namely , the **TokenizerMapper** class that defines mapper function (**map**) and the **IntSumReducer** class which implements the reducer function (**reduce**).

```
public class WordCount {
```

- Following is the **TokenizerMapper** inner class in the parent WordCount class:

```
// Mapper Class
public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    // Mapper function (map) implementation
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        StringTokenizer itr = new StringTokenizer(value.toString());

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

- The Mapper's job is to take raw input data and transform it into intermediate key-value pairs.
 - The **TokenizerMapper** class, extends **Mapper<Object, Text, Text, IntWritable>**: This defines the input and output types. It takes a generic key (usually byte offset) and a line of text, and outputs a word (Text) and a count of 1 (IntWritable).
 - The code **StringTokenizer itr = new StringTokenizer(value.toString());**, converts the line of text into individual words (tokens).
 - The code **while (itr.hasMoreTokens())**: loops through every word found in the line:
 - ◆ **word.set(itr.nextToken());**: It stores the current word in a Hadoop-optimized Text object.
 - ◆ **context.write(word, one)**: This is the crucial part. It emits a pair like ("hello", 1). If "hello" appears three times in a line, it emits this pair three times.
- Following is the **IntSumReducer** inner class in the parent WordCount class.

```
// Reducer Class
public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    // Reducer function (reduce) implementation
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

- The Reducer gathers all the 1s emitted by the Mappers for a specific word and sums them up.
 - The **IntSumReducer** class, extends **Reducer<Text, IntWritable, Text, IntWritable>**: This defines the input and output types. It takes Input Key for the word being processed, Input Value for The counts received from Mappers (e.g., a list of 1s), the Output Key for The final word to be saved to the file. and finally Output Value for the final aggregated sum (e.g., 5).
 - In the code **reduce(Text key, Iterable<IntWritable> values, ...)**, notice that the input is a key and an Iterable of values. Hadoop automatically groups all values for the same key. For example, the Reducer receives: ("hello", [1, 1, 1]).
 - The code **for (IntWritable val : values) { sum += val.get(); }** loops through the list of ones and adds them together.
 - The code line **context.write(key, result)**, writes the final result to the output file (e.g., "hello", 3).

- Following is the main method of the **WordCount** class, that drives entire map reduce process.

```
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);

    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

- Copy/Paste the above code snippets into WordCount.java that is already opened.
- The **main** method is where the Hadoop job is configured and tell the cluster how to run it.
 - **Job job = Job.getInstance(conf, "word count");**: Creates a new job object with a descriptive name.
 - **job.setMapperClass(...)** and **job.setReducerClass(...)**: Tells Hadoop which classes to use for the Map and Reduce phases.
 - **job.setCombinerClass(IntSumReducer.class)**: This is an optimization. A "Combiner" is a mini-reducer that runs on the Mapper node to shrink the data before it's sent over the network.
 - **job.setOutputKeyClass / ValueClass**: Defines the data types for the final output.
 - **FileInputFormat.addInputPath / FileOutputFormat.setOutputPath**: Specifies where your raw data is located (usually in HDFS) and where the results should be saved.
 - **job.waitForCompletion(true)**: Submits the job to the cluster and waits for it to finish, printing progress to the console.

4. Steps to Compile WordCount.java & Build the Jar file (Windows):

- Go to the command window and set the the HADOOP CLASSPATH environment variable as follows. This is to copy all the jar file directories used by hadoop in to **HADOOP_CLASSPATH** environment variable:
 - `for /f "delims=" %i in ('hadoop classpath') do set HADOOP_CLASSPATH=%i`
- Compile the **WordCount.java** program using the class path set above
 - `javac -classpath %HADOOP_CLASSPATH% WordCount.java`
- Create **wordcount.jar** using all the java classes created by compiling **WordCount.java**
 - `jar -cvf wordcount.jar *.class`

5. Starting Hadoop & Initialization Steps

- Start HDFS & YARN services by running following commands at the command prompt
 - **start-dfs.cmd**: This command starts DFS service. Wait until 2 command windows opened as part of executing this command goes to wait mode (I.e. stop spitting logs)
 - **start-yarn.cmd**: This command starts YARN service. Wait until 2 command windows opened as part of executing this command goes to wait mode (I.e. stop spitting logs)
- Create an input directory in HDFS and upload a sample text file (e.g., input.txt).
 - Run the following command at the command prompt to create **/user/wordcount/input** folder in HDFS:
 - ◆ `hdfs dfs -mkdir -p /user/wordcount/input`

- Create a text file named ***input.txt*** in the current directory and type some words in there and save. Here is an example text of words
 - ◆ ***Hello World Hello Bharat Hello Telengana Hello Hyderabad***
- Upload input.txt file to HDFS by running following command
 - ◆ ***hdfs dfs -put input.txt /user/wordcount/input***

6. Running the WordCount Map-Reduce Program in Hadoop

- Run the following command at the command prompt
 - ***hadoop jar wordcount.jar WordCount /user/wordcount/input /user/wordcount/output***
- Run following command at the command prompt to view the list files and output from the files
 - ***hdfs dfs -ls /user/wordcount/output***
 - ***hdfs dfs -cat /user/wordcount/output/part-r-00000***

Week 10 & 11: Data Processing Tool – Hive

1. What is Hive?

- **Apache Hive is a data warehouse tool built on top of Hadoop.** It allows users **to query big data** using SQL-like language (HiveQL). It converts queries into MapReduce, Tez, or Spark jobs internally.

2. Brief History

- Hive was developed by Facebook around 2007 and later contributed to the Apache Software Foundation. It was designed to help SQL users work with Hadoop easily.

3. Key Components

- HiveQL (SQL-like language), Metastore (stores metadata), Driver (manages execution), Execution Engine (runs jobs on Hadoop).

4. How Hive Works

- a) User writes SQL query
- b) Hive converts it into Hadoop jobs
- c) Hadoop processes data in HDFS
- d) Results are returned

5. Basic Commands

Component	Commands
Database	CREATE DATABASE mydb; USE mydb; SHOW DATABASES;
Table:	CREATE TABLE student(id INT, name STRING); SHOW TABLES; DESCRIBE student; DROP TABLE student;
Data Load From File	LOAD DATA INPATH '/input/file.txt' INTO TABLE student; <u>Queries:</u> ● SELECT * FROM student; ● SELECT name FROM student WHERE id=1;

6. Important Concepts

- Schema-on-Read, Partitioning, Bucketing, Managed vs External Tables.

7. Advantages

- Easy for SQL users, Handles large datasets, Scalable, Cost-effective, Good for batch processing.

8. Disadvantages

- Slow, Not suitable for real-time queries, High latency, Limited updates.

9. Uses

- Used for Data warehousing, Log analysis, Batch processing, Reporting.

10. Hive vs RDBMS

- Hive: Used for big data; Slow in processing, schema applied for reads; Useful for analytics.
- RDBMS: Used for medium data, Fast in processing, schema required for writes; Used for transactions

11. Key Points

- Hive is not a database rather it is tool for extracting data from Hadoop
- Hive uses lightweight, & embedded **Apache Derby** (Java base RDBMS)
- Hive's default database is "**metastore database**"
- Hive uses SQL like language known as HiveQL
- Hive is best for batch analytics

Installing Hive on Hadoop (Windows):

1. *Install Hadoop*

- Please see week 7 experiment for instructions on how to install hadoop

2. *Downloading Hive: This step is to download main Hive binary package.*

- Go to <https://archive.apache.org/dist/hive/hive-3.1.3>
- Download **apache-hive-3.1.3-bin.tar.gz**

3. *Move File to C:\hive : This step is to keep installation organized*

- Move the above file to C:\hive (create this directory if not present)

4. *Open a command window*

- Open a command window in administrator mode

5. *Extract Files : This step is to extract\ Hive binaries*

- Run following command: **tar -xvzf apache-hive-3.1.3-bin.tar.gz**
- Close the command window

6. *Set Environment Variable : This step is to let the system to locate Hive root directory*

- Set **HIVE_HOME** system environment variable to **C:\hive\apache-hive-3.1.3-bin**

7. *Update PATH : This step is to allow access to Hive commands globally*

- Add **%HIVE_HOME%\bin** to **PATH** system environment variable.

8. *Fix Guava Conflict (Part 1) : This step is to avoid version conflicts between hive & hadoop*

- Go to **%HIVE_HOME%\lib** directory
- Find the **guava-XX.X-jre.jar** file (**X** indicate a digit part of the version number)
- Rename it as **guava-XX.X-jre.jar.old**
- Copy **guava-32.0.1-jre.jar** from **%HADOOP_HOME%\share\hadoop\common\lib**
- Paste it in to **%HIVE_HOME%\lib**.

9. *Download Commons Collections : This step is to avoid compatibility issues*

- Go to <https://repo1.maven.org/maven2/commons-collections/commons-collections/3.2.2/>
- Download **commons-collections-3.2.2.jar** from Maven repository
- Go to **%HIVE_HOME%\lib** directory
- Rename **commons-collections4-4.4.jar** as **commons-collections4-4.4.jar.old**
- Move **commons-collections-3.2.2.jar** to **%HIVE_HOME%\lib** directory

10. *Add Windows CMD Scripts: This step enables Hive commands in Windows*

- Go to <https://github.com/HadiFadl/Hive-cmd/tree/master/bin>
- Download all ".cmd" files and save it in your **%HIVE_HOME%\bin**
- Go to <https://github.com/HadiFadl/Hive-cmd/tree/master/bin/ext>
- Download all ".cmd" files and save it in your **%HIVE_HOME%\bin\ext**
- Go to <https://github.com/HadiFadl/Hive-cmd/tree/master/bin/ext/util>
- Download all ".cmd" files and save it in your **%HIVE_HOME%\bin\ext\util**

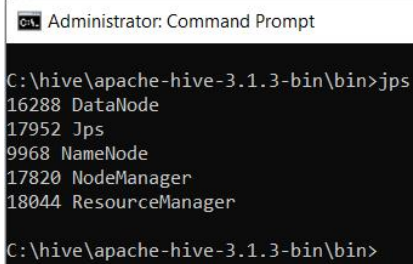
11. *Create & Configure hive-site.xml: This step is to define database and warehouse settings*

- Create a **hive-site.xml** file in **%HIVE_HOME%\conf**
- Copy following xml data into the file

```
<configuration>
  <property>
    <name>javax.jdo.option.ConnectionURL</name>
    <value>jdbc:derby;;databaseName=metastore_db;create=true</value>
  </property>
  <property>
    <name>hive.metastore.warehouse.dir</name>
    <value>/user/hive/warehouse</value>
  </property>
  <property>
    <name>hive.metastore.uris</name>
    <value></value>
  </property>
</configuration>
```

12. Start Hadoop Services : This step starts hadoop services

- Open a command window in administrator mode
- Go to %HIVE_HOME%\bin directory → **This is very important**
- start DFS service
 - Run **start-dfs.cmd** at the command prompt
- Wait until **DFS service** starts
- Start YARN Service
 - Run **start-yarn.cmd** at the command prompt
- Wait until **YARN service** starts
- Run following at the command prompt to make sure both services are running :**jps**
 - Typical out put out be as follows



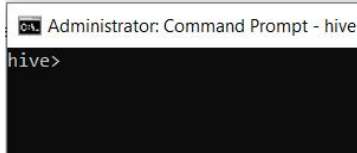
```
Administrator: Command Prompt
C:\hive\apache-hive-3.1.3-bin\bin>jps
16288 DataNode
17952 Jps
9968 NameNode
17820 NodeManager
18044 ResourceManager
C:\hive\apache-hive-3.1.3-bin\bin>
```

13. Initialize Metastore: This step is to set up Hive metadata database (First time thing)

- Run following command at the command prompt: **schematool -dbType derby -initSchema**

14. Start Hive: This step is to start the Hive

- Run following commands in the command prompt: **hive**
- After few seconds of processing **hive>** prompt should be visible



```
Administrator: Command Prompt - hive
hive>
```

- Type **<Ctrl+L>** to clear the screen

Experiments 10 & 11

Problem: Identifying properties of a data set in hive

We have a table '**user_data**' that contains the following fields:

```
data_date: string
user_id: string
properties: string
```

The properties field is formatted as a series of **<attribute>= <value>** pairs.

- Ex: Age=21; state=CA; gender=M;

Lab Instructions:

1. Create the table in HIVE using hive NoSQL based query.
2. Fill the table with sample data by using some sample data bases.
3. Write a program that produces a list of properties with minimum value(min_value), largest value(max_value) and number of unique values. Before you start, execute the prepare step to load the data into HDFS.
4. Generate a count per state.
5. Now that properties are extracted, calculate the number of records per state.

1. Create the table in HIVE using hive NoSQL based query:

- At the **hive>** prompt copy paste the following and press enter:

```
Create Table in Hive
CREATE TABLE user_data (
  data_date STRING,
  user_id STRING,
  properties STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ';'
STORED AS TEXTFILE;
```

2. Fill the table with sample data by using some sample data bases:

- Create a local file: **user_data.txt** and copy/past following:

```
2026-01-01,U1,Age=21;state=CA;gender=M
2026-01-02,U2,Age=25;state=NY;gender=F
2026-01-03,U3,Age=30;state=CA;gender=M
2026-01-04,U4,Age=22;state=TX;gender=F
2026-01-05,U5,Age=25;state=NY;gender=M
```

- Load the above file into HDFS (prepare step)
 - Type following command at the **hive>** prompt : **exit;**
 - When control goes back to command prompt, type following commands:
 - ◆ **hdfs dfs -mkdir -p /user/hive/userdata**
 - ◆ **hdfs dfs -put user_data.txt /user/hive/userdata/**
 - ◆ **hive**
 - When control goes back to command **hive>**, type following commands:

```
LOAD DATA INPATH '/user/hive/userdata/user_data.txt'
INTO TABLE user_data;
```

3. List properties with minimum value, largest value and number of unique values:

3.1. Create exploded view of properties

```
CREATE VIEW user_props AS
SELECT
    data_date,
    user_id,
    split(prop, '=')[0] AS key,
    split(prop, '=')[1] AS value
FROM user_data
LATERAL VIEW explode(split(properties, ',')) t AS prop;
```

3.2. Find min, max, unique values for each property:

```
SELECT
    key,
    MIN(value) AS min_value,
    MAX(value) AS max_value,
    COUNT(DISTINCT value) AS unique_values
FROM user_props
GROUP BY key;
```

3.3. Since values are STRING, numeric fields like Age may need casting:

```
SELECT
    key,
    MIN(CAST(value AS INT)) AS min_value,
    MAX(CAST(value AS INT)) AS max_value,
    COUNT(DISTINCT value) AS unique_values
FROM user_props
WHERE key = 'Age'
GROUP BY key;
```

4. Extract State and Count per State:

```
SELECT
    value AS state,
    COUNT(*) AS state_count
FROM user_props
WHERE key = 'state'
GROUP BY value;
```

5. Number of Records per State (Final Requirement):

```
SELECT
    state,
    COUNT(*) AS total_records
FROM (
    SELECT
        user_id,
        MAX(CASE WHEN key='state' THEN value END) AS state
    FROM user_props
    GROUP BY user_id
) s
GROUP BY state;
```

Week 12 & 13: MongoDB

Version Check:

Open cmd, and type 'mongosh'. MongoDB-shell will be opened. Run the 'db.version()' command. This will display the version of the MongoDB server you are connected to.

```
test> db.version()
```

Database Operations:

Create a database and explore basic database commands. To see all available databases, in your terminal type show dbs.

```
test> show dbs
```

After connecting to your database using mongosh, you can see which database you are using by typing db in your terminal. Create a new database called "blog"

```
test> use blog
```

You can create a collection using the createCollection() database method.

```
blog> db.createCollection("posts")
```

To insert a single document, use the insertOne() method

```
blog> db.posts.insertOne({
... title: "Post Title 1",
... body: "Body of post.",
... category: "News",
... likes: 1,
... tags: ["news", "events"],
... date: Date()
... })
```

To insert multiple documents at once, use the insertMany() method.

```
blog> db.posts.insertMany([
... {
... title: "Post Title 2",
... body: "Body of post.",
... category: "Event",
... likes: 2,
... tags: ["news", "events"],
... date: Date()
... },
... {
... title: "Post Title 3",
... body: "Body of post.",
... category: "Technology",
... likes: 3,
... tags: ["news", "events"],
... date: Date()
... },
... {
```



```

... title: "Post Title 4",
... body: "Body of post.",
... category: "Event",
... likes: 4,
... tags: ["news", "events"],
... date: Date()
... }
... ])

```

To query, or filter, data we can include a query in our find() or findOne() methods.

```
blog> db.posts.find( {category: "News"} )
```

The updateOne() method will update the first document that is found matching the provided query.

```
blog> db.posts.updateOne( { title: "Post Title 1" }, { $set: { likes: 2 } } )
```

The deleteOne() method will delete the first document that matches the query provided. The deleteMany() method will delete all documents that match the query provided.

```
blog> db.posts.deleteOne({ title: "Post Title 5" })
```

Explore data types in MongoDB

In MongoDB, a NoSQL database, data is stored in a flexible, JSON-like format known as BSON (Binary JSON). MongoDB supports a variety of data types for different kinds of information. Here are some common data types used in MongoDB:

String: Used to store textual data. Strings are UTF-8 encoded.

```

{
  "name": "John Doe"
}

```

Integer: Represents 32-bit or 64-bit signed integers.

```

{
  "age": 25
}

```

Double: Represents 64-bit floating-point numbers.

```

{
  "price": 19.99
}

```

Boolean: Represents true or false values.

```

{
  "isStudent": true
}

```

Arrays: Used to store arrays or lists of values.

```

{
  "colors": ["red", "green", "blue"]
}

```

Object: Represents an embedded document.

```

{
  "address": {
    "street": "123 Main St",
    "city": "Cityville"
  }
}

```

ObjectId: A unique identifier for documents in a collection. MongoDB automatically generates these.

```
{
  "_id": ObjectId("5fd07c8b26fb3760b4da33d1"),
  "name": "Alice"
}
```

Date: Represents date and time. Stored as milliseconds since the Unix epoch.

```
{
  "createdAt": ISODate("2022-01-01T12:00:00Z")
}
```

Null: Represents a null or non-existent value.

```
{
  "field": null
}
```

Regular Expression: Represents a JavaScript regular expression.

```
{
  "pattern": /^example.*regex$/
}
```